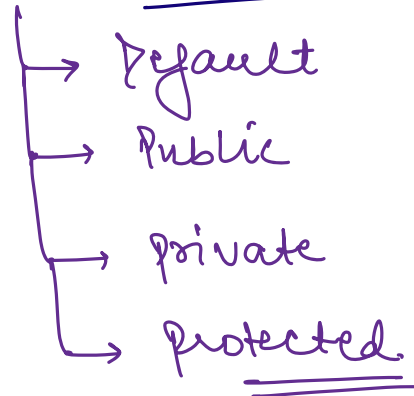


Agenda.

- Intro to OOP.
- Classes & Objects
- References
- Access Modifiers.



Intro to OOP.

C language $\Rightarrow$ Procedural.

↓
Procedure

↳ Set of instructions |
fun or method.

```
#1() {  
    ==  
    #2()  
}  
  
#2() {  
    ==  
    ==  
    ==  
    ==  
}  
  
#3() {  
    ==  
    ==  
    ==  
    ==  
    ==  
}  
  
main() {  
    #1();  
    ==  
    ==  
}
```

⇒ Deepak is teaching
Vishal is attending the class.

⇒ struct Student {
 name
 age
 batch
 psp
};

2

printStudentDetails (Student) {

 print (Student.name);

 print (Student.age);

 print (Student.batch);

 print (Student.psp);

};

3

→ print fun¹ is being applied on Student.

⇒ Student is printing their details.

OOP.

Class Student {

name

age

batch

psp

AttendClass () {

=====

}

printDetails () {

=====

}

}

st.printDetails();

Student is printing the details.

OOP.

↳ 1 principle $\Rightarrow$ Abstraction.

&

3 pillars.

Encapsulation

Inheritance

Polymorphism.

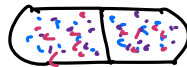
Abstraction.

$\Rightarrow$ Hiding unnecessary details from the user.

$\Rightarrow$ Just knowing the necessary details.

Encapsulation.

↳ Capsule.



Class

1) Holding all the medicines together.

2) Protecting all the medicines.

Access Modifiers.

$\Rightarrow$ Class : Blueprint of Entity.

Attrs | Fields

Behaviours. | Actions.

Class Student {

- name

- age

- batch

- psp

—
—
—

attendClass() {

—
—
—

}

bookMockInterview() {

—
—
—

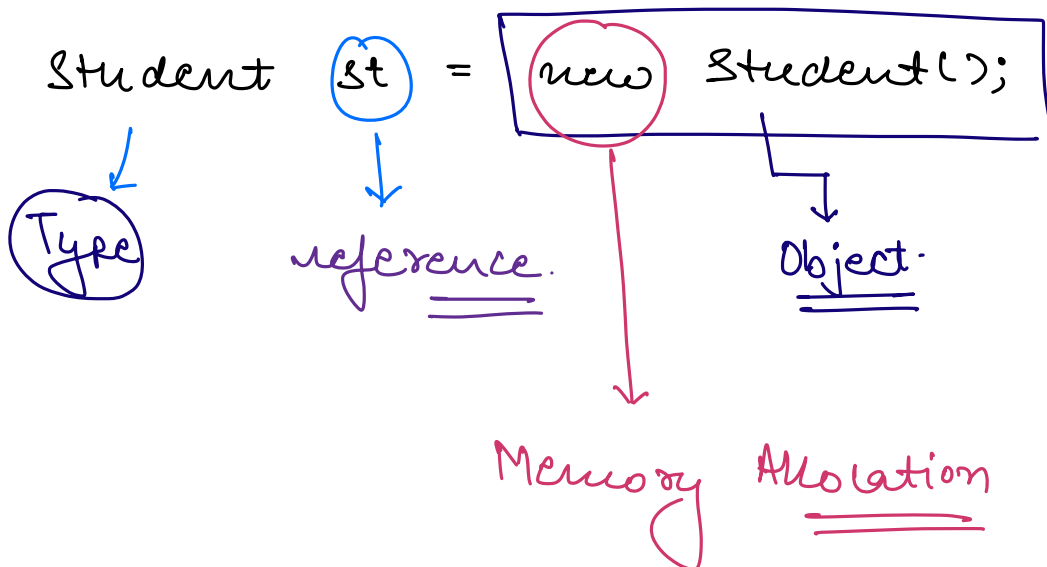
}

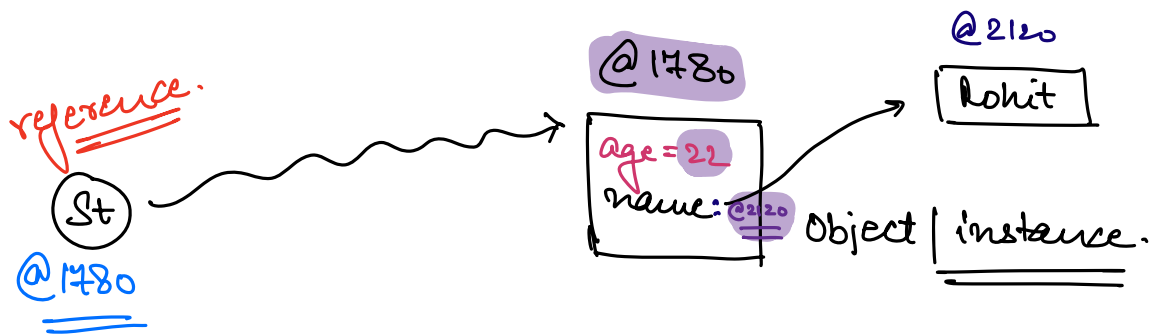
—
—
—

3

int x = 10;

⇒





$St.age = 22;$

$@1780.age = \text{—}$

$St.name = \text{"Rohit"}$

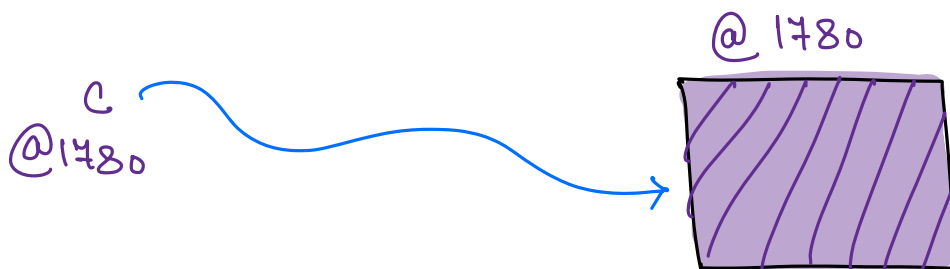
$St.attendClass()$

$St.bookMockInterview()$

$\rightarrow St$ is performing an action

$\Rightarrow Car\ C = new\ Car();$

↓
Allocates
Memory



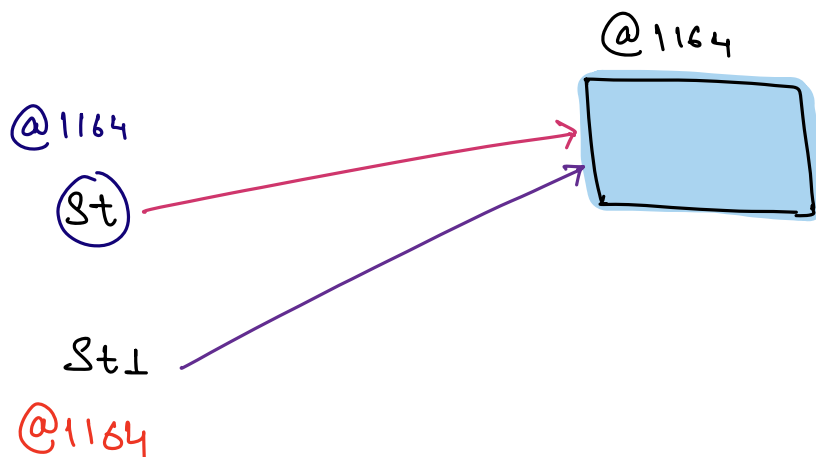
Car c = null;

```
if(—) {  
    c = —;  
}  
else if( ) {  
    —  
    —  
    —  
}
```

Student st = new ~~Car()~~;

Student st = new Student();

Student st1 = st;
@1164.



Student st = new Student();

st.age = 22;

Student st1 = st;

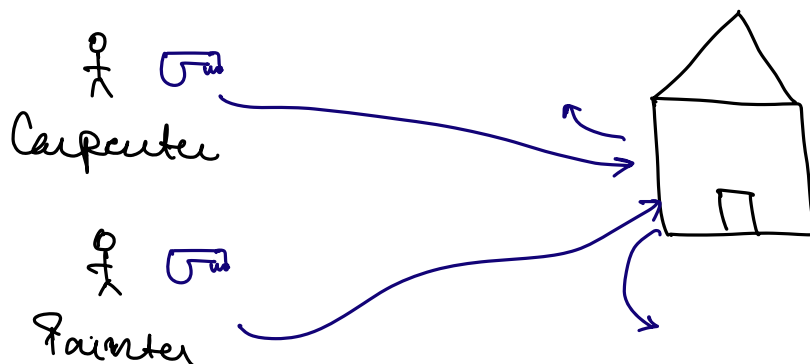
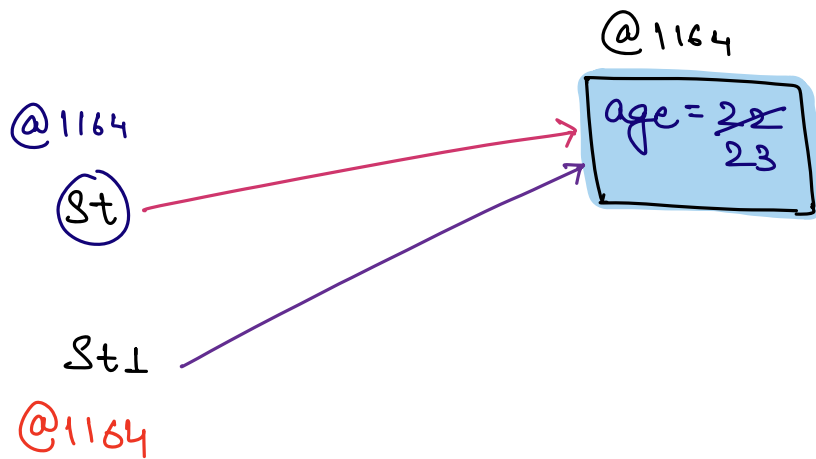
st1.age = 23;

cout(st.age) → 23

cout(st1.age) → 23

22	23
23	<u>23</u>

✓

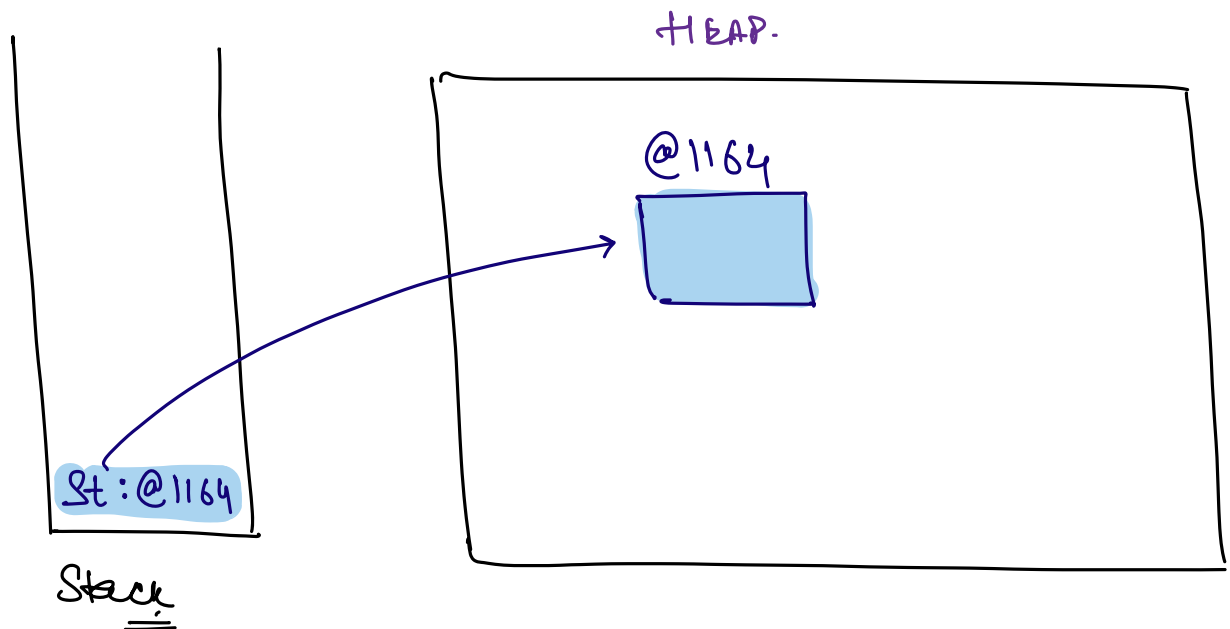
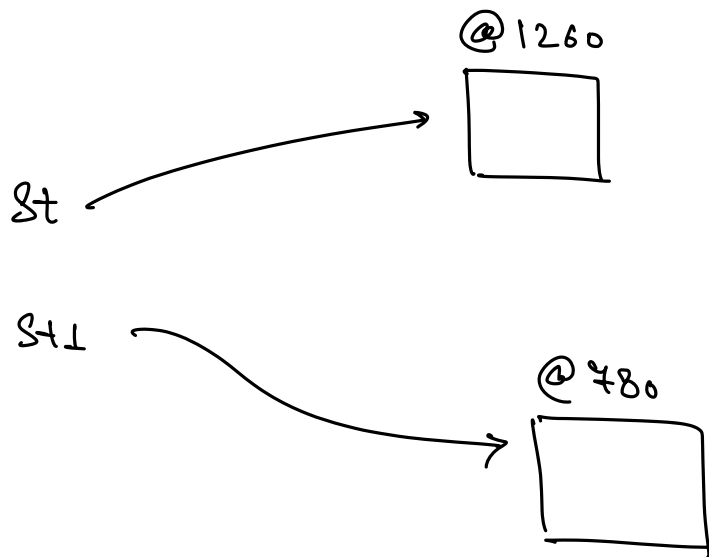


Student **st** = new Student (); → **Heap**
Constructor.

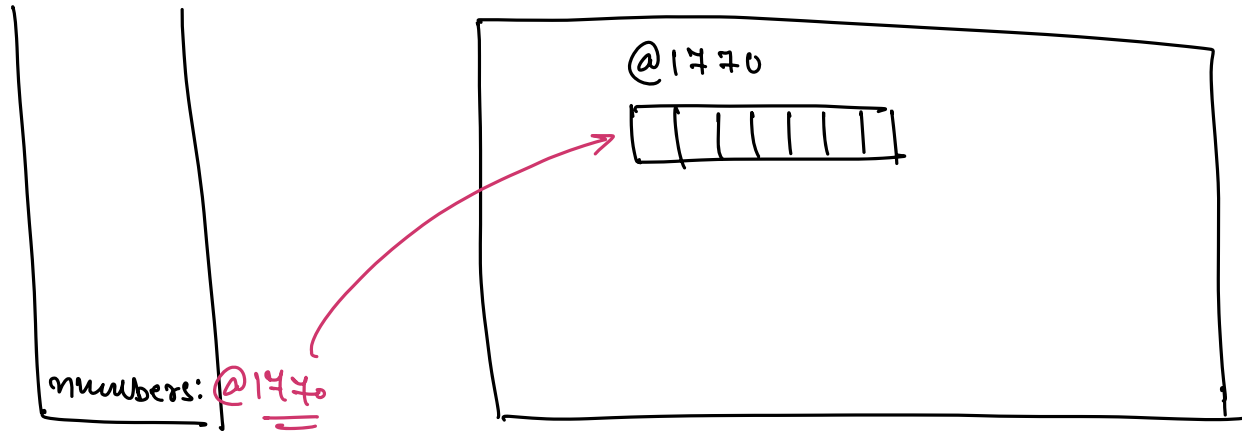
Student **st1** = new Student ();

st.age = _____;

st1.age = _____;



list < Integer > numbers = $\longleftrightarrow$;



———— * ————